

База данных DreamHome

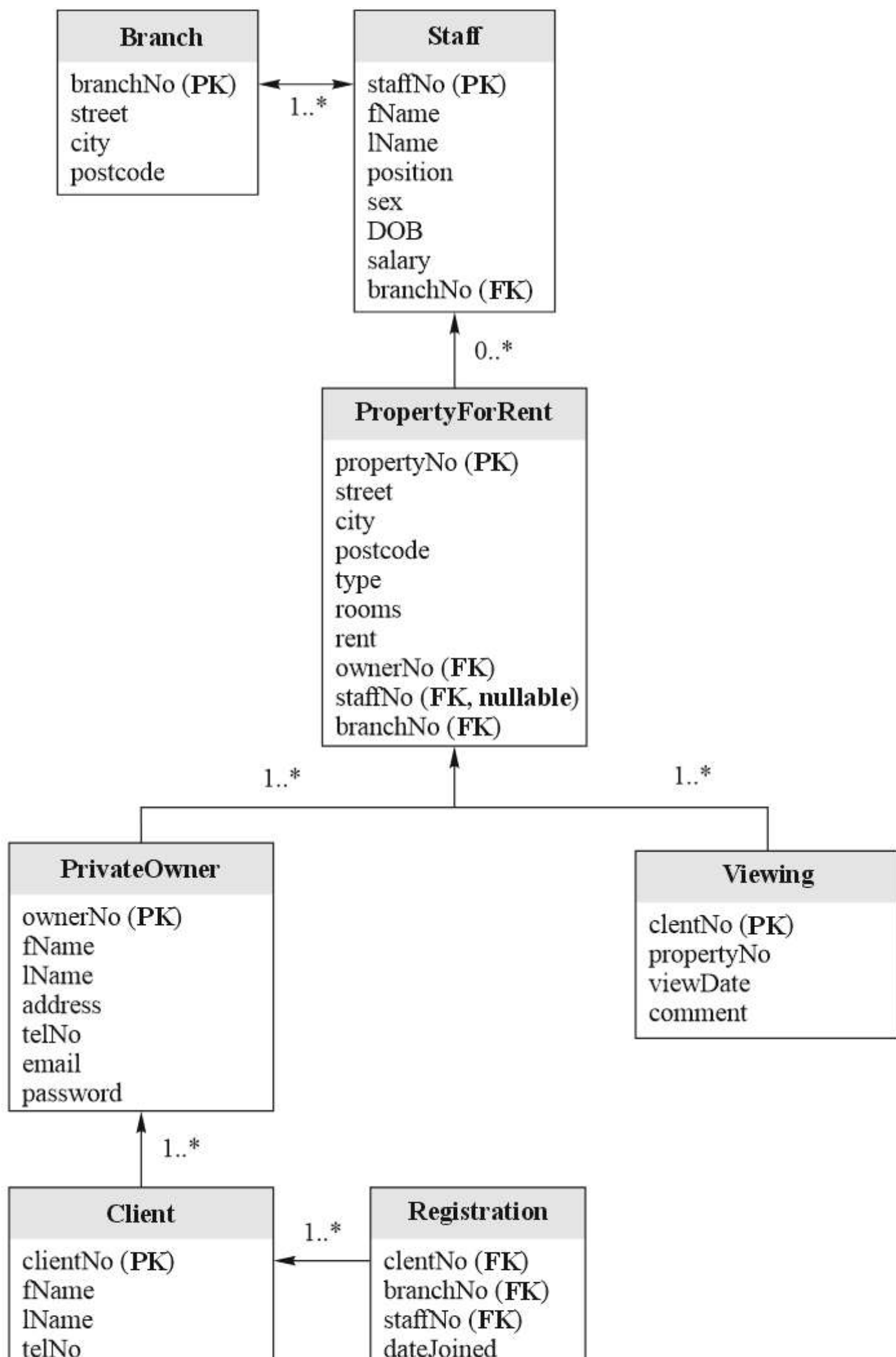
Инструкция для Windows

```
set PATH="Путь к папке с проектом";%PATH%
cd "Путь к папке с проектом"
sqlite3 DreamHome.db
.headers on
.mode column
```

cmd

```
Путь к папке с проектом
├─ sqlite3.exe
└─ DreamHome.db
```

filestructure



```
prefType
maxRent
email
```

Рис. 5.1. Структура базы данных DreamHome

5.3.1. Простые запросы

Назначение оператора `SELECT` состоит в выборке и отображении данных одной или более таблиц базы данных. Это исключительно мощный оператор, способный выполнять действия, эквивалентные операторам реляционной алгебры выборки, проекции и соединения, причем в пределах единственной выполняемой команды. Оператор `SELECT` является чаще всего используемой командой языка SQL. Общий формат оператора `SELECT` имеет следующий вид:

```
SELECT          [DISTINCT | ALL] { * | [columnExpression [AS newName]] [, ...] }      sql
FROM            TableName [alias] [, ...]
[WHERE          condition]
[GROUP BY      columnList] [HAVING condition]
[ORDER BY      columnList]
```

Здесь параметр `columnExpression` представляет собой имя столбца или выражение из нескольких имен. Параметр `TableName` является именем существующей в базе данных таблицы (или представления), к которой необходимо получить доступ. Необязательный параметр `alias` — это сокращение, устанавливаемое для имени таблицы `TableName`.

Обработка элементов оператора `SELECT` выполняется в следующей последовательности.

- `FROM` Определяются имена используемой таблицы или нескольких таблиц.
- `WHERE` Выполняется фильтрация строк объекта в соответствии с заданными условиями.
- `GROUP BY` Образуются группы строк, имеющих одно и то же значение в указанном столбце.
- `HAVING` Фильтруются группы строк объекта в соответствии с указанным условием.
- `SELECT` Устанавливается, какие столбцы должны присутствовать в выходных данных.
- `ORDER BY` Определяется упорядоченность результатов выполнения оператора. Порядок конструкций в операторе `SELECT` не может быть изменен. Только две конструкции оператора — `SELECT` и `FROM` — являются обязательными, все остальные конструкции могут быть опущены.

Операция выборки с помощью оператора `SELECT` является замкнутой, в том смысле, что результат запроса к таблице также представляет собой таблицу. Существует множество вариантов использования данного оператора, что иллюстрируется приведенными ниже примерами.

Выборка всех строк

Пример 5.1. Выборка всех столбцов и всех строк

Составьте список подробных сведений о каждом из работников. Поскольку в приведенном выше запросе не указаны никакие ограничения, в оператор не требуется помещать конструкцию WHERE. Кроме того, необходимо выбрать все существующие в таблице столбцы. Поэтому данный запрос записывается следующим образом:

```
SELECT staffNo, fName, lName, position, sex, DOB, salary, branchNo
FROM Staff;
```

sql

Поскольку выборка всех имеющихся в таблице столбцов выполняется достаточно часто, в языке SQL определен упрощенный вариант записи значения "все столбцы" - вместо имен столбцов указывается символ звездочки (*). Приведенный ниже оператор полностью эквивалентен первому и представляет собой упрощенный вариант записи того же самого запроса:

```
SELECT *
FROM Staff;
```

sql

Результат выполнения этого запроса представлен в табл. 5.1. Таблица 5.1 Результат выполнения запроса из примера 5.1

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1965-10-01	30000	B005
SG37	Ann	Beech	Assistant	F	1980-11-10	12000	B003
SG14	David	Ford	Supervisor	M	1978-03-24	18000	B003
SA9	Mary	Howe	Assistant	F	1990-02-19	9000	B007
SG5	Susan	Brand	Manager	F	1960-06-03	24000	B003
SL41	Julie	Lee	Assistant	F	1985-06-13	9000	B005

Пример 5.2. Выборка конкретных столбцов и всех строк

Создайте отчет о заработной плате всех работников с указанием только табельного номера работника (*staff No*), его имени и фамилии, а также сведений о зарплате.

```
SELECT staffNo, fName, lName, salary
FROM Staff;
```

sql

В этом примере на основе таблицы `Staff` создается новая таблица, включающая только указанные в запросе столбцы `staffNo`, `fName`, `lName` и `salary`, причем именно в этом порядке. Результат выполнения данного запроса приведен в табл. 5.2. Обратите внимание, что строки в результирующей таблице могут оказаться неотсортированными, если не дано специальное указание. С другой стороны, в некоторых СУБД сортировка результирующей таблицы выполняется по умолчанию с учетом значений одного или нескольких столбцов (например, в базе данных Microsoft Access сортировка этой результирующей таблицы была бы выполнена по значению первичного ключа `staffNo`). Способы сортировки строк в результирующей таблице рассматриваются в следующем разделе. Таблица 5.2. Результат выполнения запроса из примера 5.2

staffNo	fName	lName	salary
SL21	John	White	30000
SG37	Ann	Beech	12000
SG14	David	Ford	18000
SA9	Mary	Howe	9000
SG5	Susan	Brand	24000
SL41	Julie	Lee	9000

Пример 5.3. Использование ключевого слова `DISTINCT`

Составьте список номеров всех сдаваемых в аренду объектов, осмотренных клиентами.

```
SELECT propertyNo
FROM Viewing;
```

sql

Результат выполнения этого запроса представлен в табл. 5.3.

propertyNo
PA14

propertyNo
PG4
PG4
PA14
PG36

Обратите внимание, что результат выполнения запроса содержит повторяющиеся значения, поскольку, в отличие от операции проекции реляционной алгебры, оператор `SELECT` не исключает повторяющихся значений при выполнении проекции по значениям одного или нескольких столбцов. Для удаления из результирующей таблицы повторяющихся строк используется ключевое слово `DISTINCT`. Откорректированный запрос выглядит следующим образом:

```
SELECT DISTINCT propertyNo
FROM Viewing;
```

sql

Результаты выполнения второго варианта запроса представлены в табл. 5.4.

propertyNo
PA14
PG4
PG36

Пример 5.4. Вычисляемые поля

Создайте отчет о ежемесячной зарплате всего персонала суказанием табельного номера, имени, фамилии и суммы зарплаты.

```
SELECT staffNo, fName, lName, salary/12
FROM Staff;
```

sql

Этот запрос почти идентичен запросу из примера 5.2, за исключением того, что здесь требуется указать сумму не годовой, а ежемесячной зарплаты. В данном случае желаемый результат может быть достигнут простым делением суммы зарплаты за год на 12. Результаты выполнения запроса представлены в табл. 5.5. Таблица 5.5. Результат выполнения запроса из примера 5.4

staffNo	fName	lName	salary/12
SL21	John	White	2500
SG37	Ann	Beech	1000
SG14	David	Ford	1500
SA9	Mary	Howe	750
SG5	Susan	Brand	2000
SL41	Julie	Lee	750

Это пример использования в запросе вычисляемого поля (иногда эти поля называют расчетными, или производными). В общем случае для создания вычисляемого поля в списке **SELECT** следует указать некоторое выражение языка SQL. В этих выражениях могут применяться операции сложения, вычитания, умножения и деления. При построении сложных выражений могут использоваться круглые скобки. Для получения значения вычисляемого поля могут использоваться значения из нескольких столбцов таблицы, однако тип данных тех столбцов, которые входят в арифметические выражения, обязательно должен быть цифровым. В таблице, полученной в результате выполнения запроса, четвертый столбец называется **salary/12**. Обычно столбцам результирующей таблицы присваиваются имена соответствующих им столбцов исходных таблиц базы данных. Однако в данном случае это правило неприменимо, поскольку в стандарте SQL не определены правила именования производных столбцов. В одних диалектах языка SQL имена таким столбцам присваивают в соответствии с порядком их расположения в таблице (например, **col4**), в других диалектах у подобного столбца имя может вовсе отсутствовать или вместо него может использоваться выражение, находящееся в списке **SELECT**. Стандарт ISO позволяет явным образом задавать другие имена столбцов результирующей таблицы, для чего применяется конструкция **AS**. При использовании этой конструкции приведенный выше оператор **SELECT** может быть переписан следующим образом:

```
SELECT staffNo, fName, lName, salary/12 AS monthlySalary
FROM Staff;
```

sql

staffNo	fName	lName	monthlySalary
SL21	John	White	2500
SG37	Ann	Beech	1000
SG14	David	Ford	1500

staffNo	fName	lName	monthlySalary
SA9	Mary	Howe	750
SG5	Susan	Brand	2000
SL41	Julie	Lee	750

В этом случае четвертый столбец результирующей таблицы будет называться `monthlySalary` , а не `salary/12` .

Выборка строк (конструкция `WHERE`)

В приведенных выше примерах в результате выполнения операторов `SELECT` выбирались все строки указанной таблицы. Однако очень часто требуется тем или иным образом ограничить набор строк, помещаемых в результирующую таблицу запроса. Это достигается с помощью указания в запросе конструкции `WHERE` . Она состоит из ключевого слова `WHERE` , за которым следует перечень условий поиска, определяющих те строки, которые должны быть выбраны при выполнении запроса. Существует пять основных типов условий поиска (или предикатов, если пользоваться терминологией ISO).

- **Сравнение.** Сравниваются результаты вычисления одного выражения с результатами вычисления другого выражения.
- **Диапазон.** Проверяется, попадает ли результат вычисления выражения в заданный диапазон значений.
- **Принадлежность к множеству.** Проверяется, принадлежит ли результат вычисления выражения к заданному множеству значений.
- **Соответствие шаблону.** Проверяется, отвечает ли некоторое строковое значение заданному шаблону.
- **Значение NULL.** Проверяется, содержит ли данный столбец `NULL` (неопределенное значение).

Конструкция `WHERE` эквивалентна операции выборки реляционной алгебры. Рассмотрим примеры использования всех указанных типов условий поиска.

Пример 5.5. Условие поиска путем сравнения

Перечислите весь персонал с размером заработной платы больше 10000 фунтов стерлингов в год.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > 10000;
```

sql

В этом запросе используются таблица `Staff` и предикат `salary > 10000`. При выполнении запроса будет создана новая таблица, содержащая только те строки таблицы `Staff`, в которых значение столбца `salary` больше 10000 фунтов стерлингов. Результаты выполнения запроса представлены в табл. 5.6.

Таблица 5.6. Результат выполнения запроса из примера 5.5

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000
SG37	Ann	Beech	Assistant	12000
SG14	David	Ford	Supervisor	18000
SG5	Susan	Brand	Manager	24000

В языке SQL можно использовать простые операции сравнения, перечисленные в табл. 5.7.

Таблица 5.7. Операции сравнения

Знак операции	Назначение
=	Равно
<>	Не равно (стандарт ISO)
! =	Не равно (используется в некоторых диалектах)
<	Меньше
>	Больше
<=	Меньше или равно
>=	Больше или равно

Более сложные предикаты могут быть построены с помощью логических операций `AND`, `OR` или `NOT`, а также с помощью скобок, используемых для определения порядка вычисления выражения (если это необходимо или желательно). Вычисление выражений в условиях выполняется по следующим правилам.

- Выражение вычисляется слева направо.
- Первыми вычисляются подвыражения в скобках.
- Операции `NOT` выполняются перед операциями `AND` и `OR`.
- Операции `AND` выполняются перед операциями `OR`.

Для устранения любой возможной неоднозначности рекомендуется использовать круглые скобки.

Пример 5.6. Сложные условия поиска

Перечислите адреса всех отделений компании в Лондоне и Глазго.

```
SELECT *
FROM Branch
WHERE city = 'London' OR city = 'Glasgow';
```

sql

В этом примере для выборки сведений об отделениях компании, находящихся в Лондоне (`city = 'London'`) или Глазго (`city = 'Glasgow'`), в конструкции `WHERE` используется логический оператор `OR` . Результаты выполнения запроса представлены в табл. 5.8. Таблица 5.8. Результат выполнения запроса из примера 5.6

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B003	163 Main St	Glasgow	G11 9QX
B002	56 Clover Dr	London	NW10 6EU

Пример 5.7. Использование диапазонов (`BETWEEN / NOT BETWEEN`) в условиях поиска

Перечислите весь персонал с годовой зарплатой от 20000 до 30000 фунтов стерлингов.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary BETWEEN 20000 AND 30000;
```

sql

Наличие ключевого слова `BETWEEN` требует задания границ диапазона значений. В данном случае результаты проверки будут положительными для всех работников компании с годовой заработной платой от 20000 до 30000 фунтов стерлингов включительно. Результаты выполнения запроса представлены в табл. 5.9. Таблица 5.9. Результаты выполнений запроса из примера 5.7

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000

staffNo	fName	lName	position	salary
SG5	Susan	Brand	Manager	24000

Существует также версия проверки диапазона значений, которая имеет противоположный смысл (`NOT BETWEEN`). В этом случае требуется, чтобы проверяемое значение лежало вне границ заданного диапазона. Наличие ключевого слова `BETWEEN` и соответствующей проверки лишь незначительно повышает выразительную мощность языка SQL, поскольку те же результаты могут быть достигнуты с помощью выполнения двух обычных проверок. Приведенный выше запрос можно представить следующим образом:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary >= 20000 AND salary <= 30000;
```

sql

Однако многие полагают, что проверка принадлежности к диапазону с помощью ключевого слова `BETWEEN` является более простым способом записи условий выборки, чем обычные проверки.

Пример 5.8. Условия поиска с проверкой принадлежности к множеству (`IN / NOT IN`)

Составьте список всех руководителей и их заместителей.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position IN ('Manager', 'Supervisor');
```

sql

Проверка принадлежности к множеству обеспечивается с помощью ключевого слова `IN` . При этом проверяется, соответствует ли результат вычисления выражения одному из значений в предоставленном списке — в нашем случае это строки 'Manager' и 'Supervisor'. Результаты выполнения запроса представлены в табл. 5.10. Таблица 5.10. Результат выполнения запроса из примера 5.8

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

Существует также версия такой проверки, имеющая противоположный смысл (`NOT IN`), которая используется для отбора любых значений, кроме тех, которые указаны в представленном списке. Как и оператор `BETWEEN` , оператор `IN` незначительно повышает выразительную мощность языка SQL — тот же самый запрос может быть представлен следующим образом:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position = 'Manager' OR position = 'Supervisor';
```

sql

Однако использование оператора `IN` представляет собой более эффективный способ записи условий поиска, особенно если набор допустимых значений достаточно велик.

Пример 5.9. Условия поиска с указанием шаблонов (`LIKE` / `NOT LIKE`)

Найдите всех владельцев недвижимости, в адресах которых содержится строка `Glasgow`

При выполнении данного запроса необходимо организовать поиск строки 'Glasgow', которая может находиться в любом месте значений столбца `address` таблицы `PrivateOwner` . В языке SQL существуют два специальных символа шаблона, используемых при проверке символьных значений.

- `%` – символ процента представляет любую последовательность из нуля или более символов (поэтому часто именуется также подстановочным символом).

- `_` – символ подчеркивания представляет любой отдельный символ.

Все остальные символы в шаблоне представляют сами себя.

- `address LIKE 'H%'` – этот шаблон означает, что первый символ значения обязательно должен быть символом `H`, а все остальные символы не представляют интереса и не проверяются.

- `address LIKE 'H____'` – этот шаблон означает, что значение должно иметь длину, равную строго четырем символам, причем первым символом обязательно должен быть символ `H` .

- `address LIKE '%e'` – этот шаблон определяет любую последовательность символов длиной не менее одного символа, причем последним символом обязательно должен быть символ `e` .

- `address LIKE '%Glasgow%'` – этот шаблон означает, что нас интересует любая последовательность символов, включающая подстроку `Glasgow`;

- `address NOT LIKE 'H%'` – этот шаблон указывает, что требуются любые строки, которые не начинаются с символа `H` .

Если требуемая строка должна включать также служебный символ, обычно применяемый в качестве символа подстановки, то следует определить с помощью конструкции `ESCAPE` "маскирующий" символ, который указывает, что следующий за ним символ больше не имеет специального значения, и поместить его перед символом подстановки. Например, для проверки значений на соответствие литеральной строке `'15%'` можно воспользоваться таким предикатом:

```
LIKE '15#%' ESCAPE '#'
```

sql

С помощью механизма поиска по шаблону, существующего в языке SQL, совсем нетрудно определить всех владельцев недвижимости, проживающих в городе Глазго. Для этого достаточно выполнить следующий запрос:

```
SELECT ownerNo, fName, lName, address, telNo
FROM PrivateOwner
WHERE address LIKE '%Glasgow%';
```

sql

Результаты выполнения этого запроса представлены в табл. Таблица 5.11. Результат выполнения запроса из примера 5.9

ownerNo	fName	lName	address	telNo
CO87	Carol	Farrel	6 Achray St. Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St. Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl. Glasgow G4 0QR	0141-225-7025

Пример 5.10. Использование значения **NULL** в условиях поиска (**IS NULL** / **IS NOT NULL**)

Составьте список всех посещений сдаваемого в аренду объекта с номером **PG4** , по которым не было предоставлено комментариев. Просматривая таблицу **Viewing** , можно заметить, что в ней есть сведения о двух осмотрах сдаваемого в аренду объекта с учетным номером **PG4** , причем по одному из них комментарии представлены, а по другому — нет. На основании этого простого примера можно прийти к заключению, что вторую из этих записей можно выбрать с помощью одного из следующих условий поиска:

```
(propertyNo = 'PG4' AND comment = '')
```

sql

или

```
(propertyNo = 'PG4' AND comment <> 'too remote')
```

sql

Однако оба этих решения ошибочны. Отсутствующий комментарий (значение **NULL**) рассматривается как неопределенное значение, поэтому нельзя определить его равенство или неравенство с другой строкой. Если попробовать выполнить запрос с любым из приведенных выше условий поиска, то результирующая таблица окажется пустой.

Правильное решение состоит в явной проверке на наличие пустого значения, для чего используется специальное ключевое слово `IS NULL` :

```
SELECT clientNo, viewDate
FROM Viewing
WHERE propertyNo = 'PG4' AND comment IS NULL;
```

sql

Результаты выполнения запроса представлены в табл. 5.12. Таблица 5.12. Результат выполнения запроса из примера 5.10

clientNo	viewDate
CR56	2015-05-26

Для проверки присутствия в столбце значений, отличных от `NULL` , может использоваться версия данного условия поиска, которая имеет противоположный смысл (`IS NOT NULL`).

5.3.2. Сортировка результатов (конструкция `ORDER BY`)

В общем случае строки в результирующей таблице запроса SQL не упорядочены каким-либо определенным образом (хотя в некоторых СУБД может быть предусмотрено применение по умолчанию определенного способа упорядочения, например по первичному ключу). Однако их можно отсортировать надлежащим образом, для чего в оператор `SELECT` помещается конструкция `ORDER BY` . Конструкция `ORDER BY` включает список разделенных запятыми идентификаторов столбцов, по которым требуется упорядочить результирующую таблицу запроса. Идентификатор столбца может представлять собой либо его имя, либо номер (устарело), который обозначает элемент списка `SELECT` в соответствии с его позицией в этом списке. Самый левый элемент списка имеет номер 1, следующий — 2 и т.д. Номера столбцов могут использоваться в тех случаях, когда столбцы, по которым следует упорядочить результат, являются вычисляемыми, а конструкция `AS` с указанием имени этого столбца в операторе `SELECT` отсутствует. Конструкция `ORDER BY` позволяет упорядочить выбранные записи в порядке возрастания (`ASC`) или убывания (`DESC`) значений любого столбца или комбинации столбцов, независимо от того, присутствуют эти столбцы в таблице результатов или нет. Однако в некоторых диалектах SQL требуется, чтобы конструкция `ORDER BY` обязательно присутствовала в списке выборки оператора `SELECT` . В любом случае конструкция `ORDER BY` всегда должна быть последним элементом в операторе `SELECT` .

Пример 5.11. Сортировка по значениям одного столбца

Составьте отчет о зарплате всех работников компании, расположив строки в порядке убывания суммы зарплаты.

```
SELECT staffNo, fName, lName, salary
FROM Staff
ORDER BY salary DESC;
```

Этот пример подобен примеру 5.2. Различие состоит лишь в том, что полученные в результате выполнения запроса данные следует упорядочить по убыванию значений заработной платы в столбце `salary`. Это достигается посредством помещения в конец оператора `SELECT` конструкции `ORDER BY`, задающей сортировку результирующей таблицы по убыванию значений в столбце `salary`. А поскольку значения должны располагаться в порядке убывания, то указано ключевое слово `DESC`. Результаты выполнения запроса представлены в табл. 5.13. Обратите внимание, что конструкция `ORDER BY` может быть записана и в следующем виде: `ORDER BY 4 DESC`. Здесь `4` обозначает четвертый столбец в списке выборки оператора `SELECT`, т.е. столбец `salary`. Таблица 5.13. Результат выполнения запроса из примера 5.11

staffNo	fName	lName	salary
SL21	John	White	30000
SG5	Susan	Brand	24000
SG14	David	Ford	18000
SG37	Ann	Beech	12000
SA9	Mary	Howe	9000
SL41	Julie	Lee	9000

В конструкции `ORDER BY` может быть указано и больше одного элемента. Старший ключ сортировки определяет общую упорядоченность строк результирующей таблицы. В предыдущем примере старшим ключом сортировки является столбец `salary`. Если значения старшего ключа сортировки во всех строках результирующей таблицы являются уникальными, нет необходимости использовать дополнительные ключи сортировки. Однако, если значения старшего ключа не уникальны, в результирующей таблице будет присутствовать несколько строк с одним и тем же значением старшего ключа сортировки. В этом случае может оказаться желательным упорядочить строки с одним и тем же значением старшего ключа по какому-либо дополнительному ключу сортировки. Если в конструкции `ORDER BY` присутствуют второй и последующие элементы, то такие элементы называют младшими ключами сортировки.

Пример 5.12. Сортировка по нескольким столбцам

Подготовьте сокращенный список сдаваемых в аренду объектов, упорядоченный по типу.

```
SELECT propertyNo, type, rooms, rent
FROM PropertyForRent
ORDER BY type;
```

sql

После выполнения этого запроса будет создана результирующая таблица (табл.5.14). Таблица 5.14. Результат выполнения запроса из примера 5,12 с сортировкой по одному столбцу

propertyNo	type	rooms	rent
PL94	Flat	4	400
PG4	Flat	3	350
PG36	Flat	3	375
PG16	Flat	4	450
PA14	House	6	650
PG21	House	5	600

В полученных результатах присутствуют сведения о четырех квартирах. Если не указан младший ключ сортировки, система расположит эти строки в произвольном порядке. Для того чтобы упорядочить их, например, в порядке убывания арендной платы `rent`, следует дополнительно указать младший ключ сортировки:

```
SELECT propertyNo, type, rooms, rent
FROM PropertyForRent
ORDER BY type, rent DESC;
```

sql

Теперь результат выполнения запроса будет упорядочен вначале по типу сдаваемого в аренду объекта (по возрастанию, в алфавитном порядке) (ключевое слово `ASC` применяется по умолчанию), а в пределах одного типа объекта — в порядке убывания значений арендной платы. Результаты выполнения нового варианта запроса представлены в табл. 5.15. Таблица 5.15. Результат выполнения запроса из примера 5.12 с сортировкой по двум столбцам

propertyNo	type	rooms	rent
PG16	Flat	4	450
PL94	Flat	4	400

propertyNo	type	rooms	rent
PG36	Flat	3	375
PG4	Flat	3	350
PA14	House	6	650
PG21	House	5	600

В стандарте ISO указано, что значения `NULL` в столбцах или выражениях, для сортировки которых применяется конструкция `ORDER BY`, должны рассматриваться либо как меньшие, либо как большие по величине, чем все непустые значения. Выбор того или иного варианта оставлен на усмотрение разработчиков СУБД.

5.3.3. Использование агрегирующих функций языка SQL

Стандарт ISO содержит определение следующих пяти агрегирующих функции:

- `COUNT` — возвращает количество значений в указанном столбце;
- `SUM` — возвращает сумму значений в указанном столбце;
- `AVG` — возвращает усредненное значение в указанном столбце;
- `MIN` — возвращает минимальное значение в указанном столбце;
- `MAX` — возвращает максимальное значение в указанном столбце.

Все эти функции оперируют со значениями в единственном столбце таблицы и возвращают единственное значение. Функции `COUNT`, `MIN` и `MAX` применимы как к числовым, так и к нечисловым полям, тогда как функции `SUM` и `AVG` могут использоваться только в случае числовых полей. За исключением `COUNT(*)`, при вычислении результатов любых функций сначала исключаются все пустые значения, после чего требуемая операция применяется только к оставшимся непустым значениям столбца. Вариант `COUNT(*)` является особым случаем использования функции `COUNT` — его назначение состоит в подсчете всех строк в таблице, независимо от того, содержатся там пустые, повторяющиеся или любые другие значения. Если до применения агрегирующей функции необходимо исключить повторяющиеся значения, следует перед именем столбца в определении функции поместить ключевое слово `DISTINCT`. Стандарт ISO допускает использование ключевого слова `ALL` с целью явного указания того, что исключение повторяющихся значений не требуется, хотя это ключевое слово подразумевается по умолчанию, если никакие иные определители не заданы. Ключевое слово `DISTINCT` не имеет смысла для функций `MIN` и `MAX`. Однако его использование может оказывать влияние на результаты выполнения функций `SUM` и `AVG`, поэтому следует заранее обдумать, должно ли оно присутствовать в каждом конкретном случае. Кроме того, ключевое слово `DISTINCT` в каждом запросе может быть указано не более одного раза. Следует отметить, что агрегирующие функции могут использоваться только в списке выборки `SELECT` и в конструкции `HAVING` (см. раздел 5.3.4). Во всех других случаях применение этих функций недопустимо. Если список выборки `SELECT` содержит

агрегирующую функцию, а в тексте запроса отсутствует конструкция `GROUP BY`, обеспечивающая объединение данных в группы (см. раздел 5.3.4), то ни один из элементов списка выборки `SELECT` не может включать каких-либо ссылок на столбцы, за исключением случая, когда этот столбец используется как параметр агрегирующей функции. Например, следующий запрос является некорректным:

```
SELECT staffNo, COUNT(salary)
FROM Staff;
```

sql

Ошибка состоит в том, что в данном запросе отсутствует конструкция `GROUP BY`, а обращение к столбцу `staffNo` в списке выборки `SELECT` выполняется без применения агрегирующей функции.

Пример 5.13. Использование функции `COUNT(*)`

Определите, сколько сдаваемых в аренду объектов имеют ставку арендной платы более 350 фунтов стерлингов в месяц.

```
SELECT COUNT(*) AS count
FROM PropertyForRent
WHERE rent > 350;
```

sql

Ограничение на подсчет только тех сдаваемых в аренду объектов, арендная плата которых составляет более 350 фунтов стерлингов в месяц, реализуется посредством использования конструкции `WHERE`. Общее количество сдаваемых в аренду объектов, отвечающих указанному условию, может быть определено с помощью агрегирующей функции `COUNT`. Результаты выполнения запроса представлены в табл. 5.16. Таблица 5.16. Результат выполнения запроса из примера 5.13

count
5

Пример 5.14. Использование функции `COUNT(DISTINCT)`

Определите, сколько различных сдаваемых в аренду объектов было осмотрено клиентами в мае 2015 года.

```
SELECT COUNT(DISTINCT propertyNo) AS count
FROM Viewing
WHERE viewDate BETWEEN '2015-05-01' AND '2015-05-31';
```

sql

И в этом случае ограничение результатов запроса анализом только тех сдаваемых в аренду объектов, которые были осмотрены в мае 2001 года, достигается посредством использования конструкции `WHERE`. Общее количество осмотренных объектов, удовлетворяющих указанному условию, может быть определено с помощью агрегирующей функции `COUNT`. Однако, поскольку один и тот же объект может быть осмотрен различными клиентами несколько раз, необходимо в определении функции указать ключевое слово `DISTINCT` — это позволит исключить из расчета повторяющиеся значения. Результаты выполнения запроса представлены в табл. 5.17. Таблица 5.17. Результат выполнения запроса из примера 5.14

count
2

Пример 5.15. Использование функций `COUNT` и `SUM`

Определите общее количество менеджеров компании и вычислите сумму их годовой зарплаты.

```
SELECT COUNT(staffNo) AS count, SUM(salary) AS sum
FROM Staff
WHERE position = 'Manager';
```

sql

Ограничение на отбор сведений только о менеджерах компании достигается указанием в запросе соответствующей конструкции `WHERE`. Общее количество менеджеров и сумма их годовой заработной платы определяются путем применения к результирующей таблице запроса агрегирующих функций `COUNT` и `SUM`. Результаты выполнения запроса представлены в табл. 5.18.

count	sum
2	54000

Пример 5.16. Использование функций `MIN`, `MAX` и `AVG`

Вычислите значение минимальной, максимальной и средней заработной платы.

```
SELECT MIN(salary) AS min, MAX(salary) AS max, AVG(salary) AS avg
FROM Staff;
```

sql

В этом примере необходимо обработать сведения обо всем персонале компании, поэтому использовать конструкцию `WHERE` не требуется. Необходимые значения могут быть вычислены с помощью функций `MIN`, `MAX` и `AVG`, применяемых к столбцу `salary` таблицы

Staff . Результаты выполнения запроса представлены в табл. 5.19. Таблица 5.19. Результат выполнения запроса из примера 5.16

min	max	avg
9000	30000	17000.0

5.3.4. Группирование результатов (конструкция GROUP BY)

Приведенные выше примеры сводных данных подобны итоговым строкам, обычно размещаемым в конце отчетов. В итогах все детальные данные отчета сжимаются в одну обобщающую строку. Однако очень часто в отчетах требуется формировать и промежуточные итоги. Для этой цели в операторе **SELECT** может указываться конструкция **GROUP BY** . Запрос, в котором присутствует конструкция **GROUP BY** , называется группирующим запросом, поскольку в нем группируются данные, полученные в результате выполнения операции **SELECT** , после чего для каждой отдельной группы создается единственная итоговая строка. Столбцы, перечисленные в конструкции **GROUP BY** , называются группируемыми столбцами. Стандарт ISO требует, чтобы конструкции **SELECT** и **GROUP BY** были тесно связаны между собой. При использовании в операторе **SELECT** конструкции **GROUP BY** каждый элемент списка в списке выборки **SELECT** должен иметь единственное значение для всей группы. Более того, конструкция **SELECT** может включать только следующие типы элементов:

- имена столбцов;
- агрегирующие функции;
- константы;
- выражения, включающие комбинации перечисленных выше элементов.

Все имена столбцов, приведенные в списке выборки **SELECT** , должны присутствовать и в конструкции **GROUP BY** , за исключением случаев, когда имя столбца используется только в агрегирующей функции. Противоположное утверждение не всегда справедливо — в конструкции **GROUP BY** могут присутствовать имена столбцов, отсутствующие в списке выборки **SELECT** . Если совместно с конструкцией **GROUP BY** используется конструкция **WHERE** , то она обрабатывается в первую очередь, а группированию подвергаются только те строки, которые удовлетворяют условию поиска. Стандартом ISO определено, что при проведении группирования все отсутствующие значения рассматриваются как равные. Если две строки таблицы в одном и том же группируемом столбце содержат значения **NULL** и идентичные значения во всех остальных непустых группируемых столбцах, они помещаются в одну и ту же группу.

Пример 5.17. Использование конструкции GROUP BY

Определите количество персонала, работающего в каждом из отделений компании, а также их суммарную заработную плату.

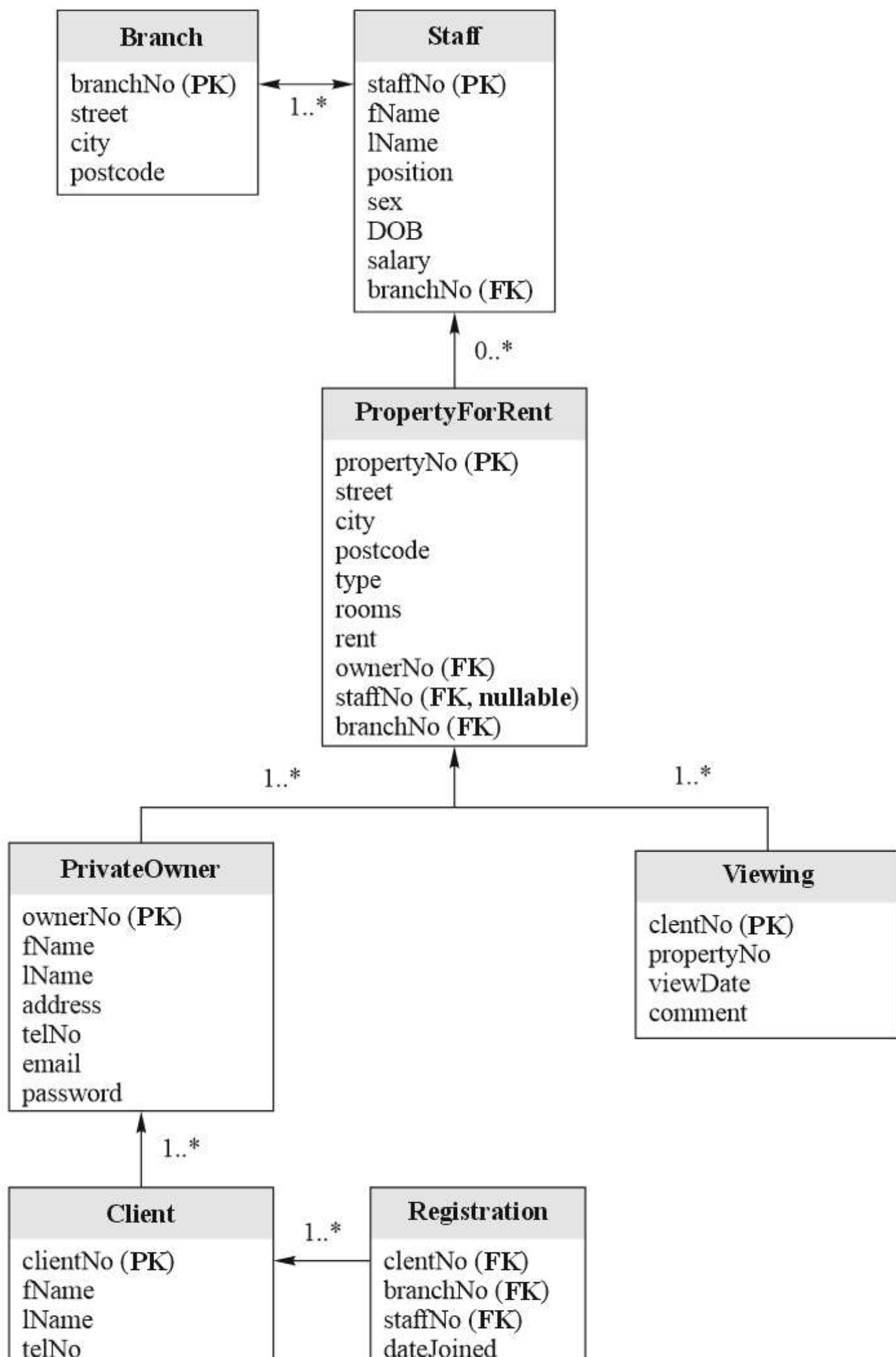
```
SELECT branchNo, COUNT(staffNo) AS count, SUM(salary) AS sum
FROM Staff
GROUP BY branchNo
ORDER BY branchNo;
```

Нет необходимости включать имена столбцов `staffNo` и `salary` в список элементов `GROUP BY`, поскольку они появляются только в списке выборки `SELECT` с агрегирующими функциями. В то же время столбец `branchNo` в списке конструкции `SELECT` не связан с какой-либо агрегирующей функцией и по этой причине обязательно должен быть указан в конструкции `GROUP BY`. Результаты выполнения запроса представлены в табл. 5.20. Таблица 5.20. Результат выполнения запроса из примера 5.17

branchNo	count	sum
B003	3	54000
B005	2	39000
B007	1	9000

Концептуально при обработке этого запроса выполняются следующие действия.

1. Строки таблицы `Staff` распределяются в группы в соответствии со значениями в столбце номера отделения компании. В пределах каждой из групп оказываются данные обо всем персонале одного из отделений компании. В нашем примере будут созданы три группы, как показано на рис. 5.2.
2. Для каждой из групп вычисляются общее количество строк, равное количеству работников отделения, а также сумма значений в столбце `salary`, которая и является интересующей нас суммой заработной платы всех работников отделения. Затем генерируется единственная итоговая строка для всей группы исходных строк.
3. Полученные строки результирующей таблицы сортируются в порядке возрастания номера отделения, указанного в столбце `branchNo`.



prefType
maxRent
email

--

Рис. 5.2. Три группы записей, создаваемые при выполнении запроса

Стандарт SQL допускает помещение в список выборки `SELECT` вложенных запросов (раздел 5.3.5). Поэтому приведенный выше запрос можно также представить следующим образом:

```

SELECT branchNo, (SELECT COUNT (staffNo) AS count
                  FROM Staff s
                  WHERE s.branchNo = b.branchNo),
               (SELECT SUM (salary) AS sum
                  FROM Staff s
                  WHERE s.branchNo = b.branchNo)
FROM Branch b
ORDER BY branchNo;

```

sql

Но в этой версии запроса для каждого из отделений компании, описанных в таблице `Branch` , создаются два результата вычисления агрегирующих функций, поэтому в некоторых случаях возможно появление строк, содержащих нулевые значения.

branchNo	(SELECT COUNT (staffNo) AS count	(SELECT SUM (salary) AS sum
B002	0	
B003	3	54000
B004	0	
B005	2	39000
B007	1	9000

Ограничения на выполнение группирования (конструкция `HAVING`)

Конструкция `HAVING` предназначена для использования совместно с конструкцией `GROUP BY` для задания ограничений, указываемых с целью отбора тех групп, которые будут помещены в результирующую таблицу запроса. Хотя конструкции `HAVING` и `WHERE` имеют сходный синтаксис, их назначение различно. Конструкция `WHERE` предназначена для отбора отдельных строк, предназначенных для заполнения результирующей таблицы запроса, а конструкция `HAVING` используется для отбора групп, помещаемых в результирующую таблицу запроса. Стандарт ISO требует, чтобы имена столбцов, применяемые в конструкции `HAVING` , обязательно присутствовали в списке элементов `GROUP BY` или применялись в

агрегирующих функциях. На практике условия поиска в конструкции `HAVING` всегда включают, по меньшей мере, одну агрегирующую функцию; в противном случае эти условия поиска должны быть помещены в конструкцию `WHERE` и применены для отбора отдельных строк. (Помните, что агрегирующие функции не могут использоваться в конструкции `WHERE`.) Конструкция `HAVING` не является необходимой частью языка SQL — любой запрос, написанный с использованием конструкции `HAVING`, может быть представлен в ином виде, без ее применения.

Пример 5.18, Использование конструкции `HAVING`

Для каждого отделения компании с численностью персонала более одного человека определите количество работающих и сумму их заработной платы.

```
SELECT branchNo, COUNT(staffNo) AS count, SUM(salary) AS sum
FROM Staff
GROUP BY branchNo
HAVING COUNT (staffNo) > 1
ORDER BY branchNo;
```

sql

Этот пример аналогичен предыдущему, но здесь используются дополнительные ограничения, указывающие на то, что нас интересуют сведения только о тех отделениях компании, в которых работает больше одного человека. Подобное требование налагается на группы, поэтому в запросе следует использовать конструкцию `HAVING`. Результаты выполнения запроса представлены в табл. 5.21. Таблица 5.21. Результат выполнения запроса из примера 5.18

branchNo	count	sum
B003	3	54000
B005	2	39000

5.3.5. Подзапросы

В этом разделе мы обсудим использование законченных операторов `SELECT`, внедренных в тело другого оператора `SELECT`. Внешний (второй) оператор `SELECT` использует результат выполнения внутреннего (первого) оператора для определения содержания окончательного результата всей операции. Внутренние запросы могут находиться в конструкциях `WHERE` и `HAVING` внешнего оператора `SELECT` — в этом случае они получают название подзапросов, или вложенных запросов. Кроме того, внутренние операторы `SELECT` могут использоваться в операторах `INSERT`, `UPDATE` и `DELETE` (см. раздел 5.3.10). Существуют три типа подзапросов.

- **Скалярный подзапрос** возвращает значение, выбираемое из пересечения одного столбца

с одной строкой, т.е. единственное значение. В принципе скалярный подзапрос может использоваться везде, где требуется указать единственное значение. Варианты скалярных подзапросов приведены в примерах 5.13 и 5.14.

- **Строковый подзапрос** возвращает значения нескольких столбцов таблицы, но в виде единственной строки. Строковый подзапрос может использоваться везде, где применяется конструктор строковых значений, — обычно это предикаты. Вариант строкового подзапроса приведен в примере 5.15.

- **Табличный подзапрос** возвращает значения одного или нескольких столбцов таблицы, размещенные в более чем одной строке. Табличный подзапрос может использоваться везде, где допускается указывать таблицу, например как операнд предиката `IN`.

Пример 5.19. Использование подзапроса с проверкой на равенство

Составьте список персонала, работающего в отделении компании, расположенном по адресу `463 Main St`.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = (
    SELECT branchNo
    FROM Branch
    WHERE street = '163 Main St'
);
```

sql

Внутренний оператор `SELECT (SELECT branchNo FROM Branch ...)` предназначен для определения номера отделения компании, расположенного по адресу `'163 Main St'`. (Существует только одно такое отделение компании, поэтому данный пример является примером скалярного подзапроса.) После получения номера требуемого отделения выполняется внешний подзапрос, предназначенный для выборки подробных сведений о работниках этого отделения. Иначе говоря, внутренний оператор `SELECT` возвращает таблицу, состоящую из единственного значения `'B003'`. Оно представляет собой номер того отделения компании, которое находится по адресу `'163 Main St'`. В результате внешний оператор `SELECT` приобретает следующий вид:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = 'B003';
```

sql

Результаты выполнения этого запроса представлены в табл. 5.22. Таблица 5.22. Результат выполнения запроса из примера 5.19

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

Подзапрос представляет собой инструмент создания временной таблицы, содержимое которой извлекается и обрабатывается внешним оператором. Подзапрос. можно указывать непосредственно после операторов сравнения (т.е. операторов `=`, `<`, `>`, `<=`, `>=`, `<>`) в конструкции `WHERE` или `HAVING`. Текст подзапроса должен быть заключен в круглые скобки.

Пример 5.20. Использование подзапросов с агрегирующими функциями

Составьте список всех сотрудников, имеющих зарплату выше средней, указав, насколько их зарплата превышает среднюю зарплату по предприятию.

```
SELECT staffNo, fName, lName, position,
       salary - (SELECT AVG (salary) FROM Staff) AS salDiff
FROM Staff
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

sql

Необходимо отметить, что нельзя непосредственно включить в запрос выражение `'WHERE salary > AVG (salary)'` поскольку применять агрегирующие функции в конструкции `WHERE` запрещено. Для достижения желаемого результата следует создать подзапрос, вычисляющий среднее значение годовой заработной платы, а затем использовать его во внешнем операторе `SELECT`, предназначенном для выборки сведений о тех работниках компании, чья зарплата превышает это среднее значение. Иначе говоря, подзапрос возвращает значение средней зарплаты по компании в год, равное **17000** фунтов стерлингов. Результат выполнения этого скалярного подзапроса используется во внешнем операторе `SELECT` как для вычисления отклонения зарплаты от среднего уровня, так и для отбора сведений о работниках. Поэтому внешний оператор `SELECT` приобретает следующий вид:

```
SELECT staffNo, fName, lName, position, salary - 17000 As salDiff
FROM Staff
WHERE salary > 17000;
```

sql

Результаты выполнения запроса представлены в табл. 5.23 Таблица 5.23. Результат выполнения запроса из примера 5.20

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000
SG14	David	Ford	Supervisor	1000
SG5	Susan	Brand	Manager	7000

К подзапросам применяются следующие правила и ограничения.

1. В подзапросах не должна использоваться конструкция `ORDER BY` , хотя она может присутствовать во внешнем операторе `SELECT` .
2. Список выборки `SELECT` подзапроса должен состоять из имен отдельных столбцов или составленных из них выражений, за исключением случая, когда в подзапросе используется ключевое слово `EXISTS` (см. раздел 5.3.8).
3. По умолчанию имена столбцов в подзапросе относятся к таблице, имя которой указано в конструкции `FROM` подзапроса. Однако разрешается ссылаться и на столбцы таблицы, указанной в конструкции `FROM` внешнего запроса, для чего используются уточненные имена столбцов (как описано ниже).
4. Если подзапрос является одним из двух операндов, участвующих в операции сравнения, то подзапрос должен указываться в правой части этой операции. Например, приведенный ниже вариант записи запроса из предыдущего примера является некорректным, поскольку подзапрос размещен в левой части операции сравнения со значением столбца `salary` .

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE (SELECT AVG (salary) FROM Staff) < salary;
```

sql

Пример 5.21 Вложенные подзапросы и использование предиката `IN`

Составьте перечень сдаваемых в аренду объектов, за которые отвечают работники отделения компании, расположенного по адресу '163 Main St' .

```
SELECT propertyNo, street, city, postcode, type, rooms, rent
FROM PropertyForRent
WHERE staffNo IN (SELECT staffNo
                  FROM Staff
                  WHERE branchNo = (SELECT branchNo
                                     FROM Branch
                                     WHERE street = '163 Main St'));
```

sql

Первый, самый внутренний, запрос предназначен для определения номера отделения компании, расположенного по адресу '163 Main St' . Второй, промежуточный, запрос осуществляет выборку сведений о персонале, работающем в этом отделении. В данном случае выбирается больше одной строки данных и поэтому во внешнем запросе нельзя использовать оператор сравнения = . Вместо него необходимо использовать ключевое слово **IN** . Внешний запрос осуществляет выборку сведений о сдаваемых в аренду объектах, за которые отвечают те работники компании, данные о которых были получены в результате выполнения промежуточного запроса. Результаты выполнения запроса представлены в табл. 5.24. Таблица 5.24. Результат выполнения запроса из примера 5.21

propertyNo	street	city	postcode	type	rooms	rent
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450

5.3.6. Ключевые слова ANY и ALL

Ключевые слова **ANY** и **ALL** могут использоваться с подзапросами, возвращающими один столбец чисел. Если подзапросу будет предшествовать ключевое слово **ALL** , условие сравнения считается выполненным только в том случае, если оно выполняется для всех значений в результирующем столбце подзапроса. Если тексту подзапроса предшествует ключевое слово **ANY** , то условие сравнения будет считаться выполненным, если оно удовлетворяется хотя бы для какого-либо (одного или нескольких) значения в результирующем столбце подзапроса. Если в результате выполнения подзапроса будет получено пустое значение, то для ключевого слова **ALL** условие сравнения будет считаться выполненным, а для ключевого слова **ANY** — невыполненным. Согласно стандарту ISO дополнительно можно использовать ключевое слово **SOME** , являющееся синонимом ключевого слова **ANY** .

Пример 5.22. Использование ключевых слов ANY и SOME

Найдите всех работников, чья зарплата превышает зарплатухотя бы одного работника отделения компании под номером 'B003' .

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > ANY (
    SELECT salary
    FROM Staff
```

sql

```
WHERE branchNo = 'B003'
);
```

В `sqlite` `ANY` и `SOME` не поддерживаются, поэтому переписываем в следующем виде:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > (
    SELECT MIN(salary)
    FROM Staff
    WHERE branchNo = 'B003'
);
```

sql

Хотя этот запрос может быть записан с использованием подзапроса, определяющего минимальную зарплату персонала отделения под номером `'B003'`, после чего внешний подзапрос сможет выбрать сведения обо всем персонале компании, чья зарплата превосходит это значение (см. пример 5.20), возможен и другой подход, заключающийся в использовании ключевых слов `SOME` / `ANY`. В этом случае внутренний подзапрос создает множество значений {12000, 18000, 24000}, а внешний запрос выбирает сведения о тех работниках, чья зарплата больше любого из значений в этом множестве (фактически больше минимального значения — 12 000). Подобный альтернативный метод можно считать более естественным, чем определение в подзапросе минимальной зарплаты. Но и в том и в ином случае вырабатываются одинаковые результаты выполнения запроса, которые представлены в табл. 5.25.

Таблица 5.25. Результат выполнения запроса из примера 5.22

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000
SG14	David	Ford	Supervisor	18000
SG5	Susan	Brand	Manager	24000

Пример 5.23. Использование ключевого слова `ALL`

Найдите всех работников, чья заработная плата больше заработной платы любого работника отделения компании под номером `'B003'`.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > ALL(SELECT salary
```

sql

```
FROM Staff
WHERE branchNo = 'B003');
```

По аналогии с предыдущим случаем, оператор 'ALL' в SQLite не поддерживается:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > (
    SELECT MAX(salary)
    FROM Staff
    WHERE branchNo = 'B003'
);
```

sql

В целом этот запрос подобен предыдущему. И в данном случае можно было бы использовать подзапрос, определяющий максимальное значение зарплаты персонала отделения под номером 'B003', после чего с помощью внешнего запроса выбрать сведения обо всех работниках компании, зарплата которых превышает это значение. Однако в данном примере выбран подход с использованием ключевого слова ALL. Результаты выполнения запроса представлены в табл. 5.26. Таблица 5.26. Результат выполнения запроса из примера 5.23

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000

В целом этот запрос подобен предыдущему. И в данном случае можно было бы использовать подзапрос, определяющий максимальное значение зарплаты персонала отделения под номером 'B003', после чего с помощью внешнего запроса выбрать сведения обо всех работниках компании, зарплата которых превышает это значение. Однако в данном примере выбран подход с использованием ключевого слова ALL. Результаты выполнения запроса представлены в табл. 5.26.